

class tableau

```
{ int t, taille, nbelts;
```

public:

```
tableau(int);
```

```
tableau(const tableau &); // tableau A = B;
```

```
~tableau();
```

```
void affiche();
```

```
int chercher(int);
```

```
void termine();
```

```
int produit_scal(const tableau &);
```

```
tableau somme(const tableau &);
```

```
int nombre_elt();
```

```
void copie();
```

```
void ajout(int);
```

```
friend void affiche1(const tableau &);
```

```
};
```

```
tableau::tableau(int a = 20)
```

```
{ taille = a;
```

```
t = new int[taille];
```

```
nbelts = 0;
```

```
}
```

```
tableau::tableau(const tableau &v)
```

```
{ taille = v.taille;
```

```
t = new int[taille];
```

```
nbelts = v.nbelts;
```

```
for (int i = 0; i < nbelts; i++)
```

```
{ t[i] = v.t[i];
```

```
}
```

tableau B(5);

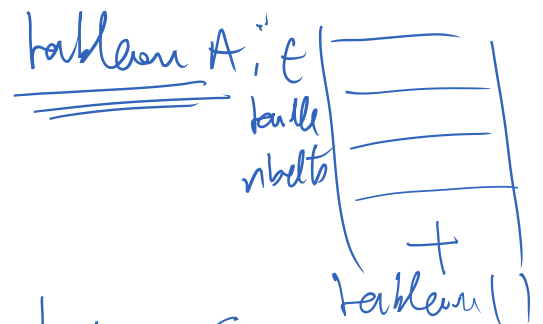
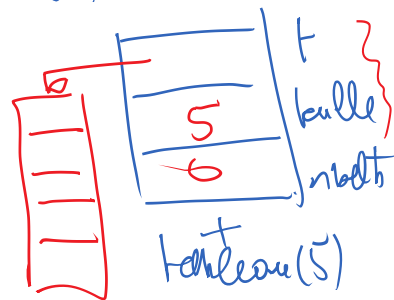


tableau C = B;

```

{
tableau :: n tableau()
} delete t;

```

```

{
void tableau::affiche()
{
for (int i=0; i < nbelt; i++)
{ cout << t[i] << "t";

```

```

;
cout << "\n";
}

```

```

int tableau::chercher (int x)
{
int i=0;
while ( i < nbelt && t[i] != x)
{ i++;
if ( i == nbelt)
return -1;
else
return i;
}

```

```

void tableau::termine5( )
{
for (int i=0; i < nbelt; i++)
{ if ( t[i] % 10 == 5)
{ cout << t[i] << "t";
;
}
}
}

```

```

void tableau::ajout (int a)
{
if (nbelt == taille)
cout << "Pas de place\n";
else
{ t[nbelt] = a; // t[nbelt++] = a;
nbelt++;
}
}

```

// tableau::nbelt

```
int tableau::produit_scal(const tableau &v)
```

```
{ if (nbelts == v.nbelts)
```

```
    int som = 0;
```

```
    for (int i = 0; i < nbelts; i++)
```

```
    { som = som + H[i] * v.H[i];
```

```
    }
```

```
    return som;
```

```
    else
```

```
    {
```

```
        cout << "Deux objets n'ayant pas le même nombre d'elts";
```

```
    }
```

```
int tableau::nombre_elt()
```

```
{ return nbelts; }
```

```
tableau tableau::somme(const tableau &v)
```

```
{ tableau res (taille);
```

```
  for (int i = 0; i < nbelts; i++)
```

```
  { res.H[i] = H[i] + v.H[i];
```

```
  }
```

```
  res.nbelts = nbelts;
```

```
  return res;
```

```
void tableau::saisie()
```

```
{ do cout << "Entrez le nombre d'elts : ";
```

```
  cin >> nbelts;
```

```
  while (nbelts <= 0);
```

```
  for (int i = 0; i < nbelts; i++)
```

```
  { cout << "Entrez l'elt numero " << i+1 << " : ";
```

```
    cin >> H[i];
```

```
  }
```

main()

↳ tableau A(15);

A.saisir();

A.affiche();

tableau B=A;

① tableau *p;

② p = new tableau;

③ delete p;

int y;

cout << "Donnez un elt à
rechercher:";

cin >> y;

int p = B.chercher(y);

if (p == -1)

cout << "Element inexistant\n";

else

cout << "Element existe et sa position est" << p+1 << "\n";

cout << "les elts de A qui se terminent par 5 sont:";

A.termine_5();

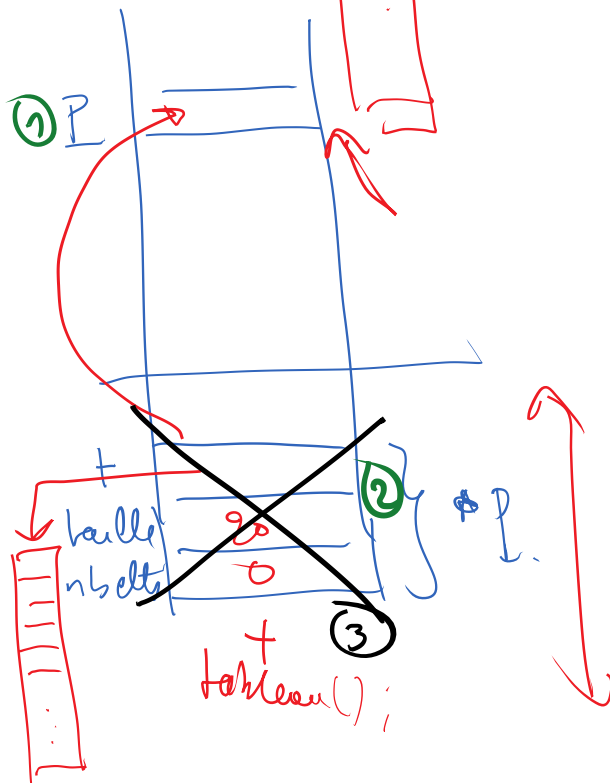
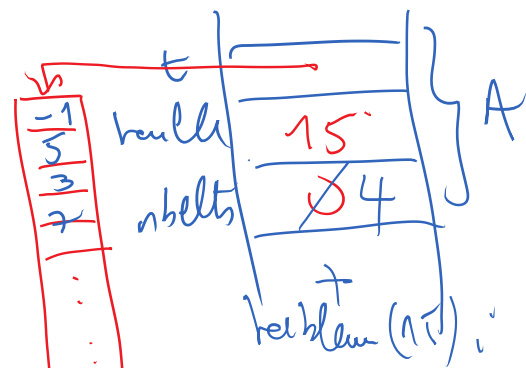
if (A.nombre_elt() == B.nombre_elt())

{ y = A.produit_scal(B);

cout << "Le produit scalaire de A et B est:" << y << "\n";

{

else } cout << "A et B n'ont pas le même nombre d'elts\n";



cout << "Entrez un elt à ajouter : ";
 cin >> y;

B. ajout(y);

```

if (A.nombre_elt() == B.nombre_elt())
{
  int p = A.nombre_elt();
  tableau S(p);

```

S = A.somme(B);

S.affiche();

else

{ cout << "somme impossible\n";

}

}

Séance :

./ fonction amie :

void affiche1(const tableau v)

{ for (int i = 0; i < v.nombre; i++)

{ cout << v.t[i] << " ";

cout << "\n";

}

Dans main : affiche1(A); // et non A.affiche1();

Exemple d'utilisation :

class B;

class A

{ ~
~
~ }

friend void MOUKTADER (const A&,
~
~
~
Y [9])

class B

{ ~
~
~ }

friend void MOUKTADER (const A&
const B&);

~
~
~
{ 9 }

type d'amitié :

- ① MOUKTADER est membre d'une classe et amie de l'autre classe.

On écrit : Dans A :

void MOUKTADER (const B&);

• Dans B

friend void A::MOUKTADER (const B&);

- ② MOUKTADER est indépendante des deux classes.

On écrit : Dans A

friend void MOUKTADER (const A&, const B&);

Dans B

friend void MOUKTADER (const A&, const B&);

→ Calcul d'une matrice par un vecteur

$$\begin{pmatrix} 3 & 0 & 1 \\ 2 & 7 & -2 \\ 5 & 8 & 4 \end{pmatrix} \begin{pmatrix} 9 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 3 \times 9 + 0 \times 1 + 1 \times 0 \\ 2 \times 9 + 7 \times 1 + (-2) \times 0 \\ 5 \times 9 + 8 \times 1 + 4 \times 0 \end{pmatrix} = \begin{pmatrix} 27 \\ 25 \\ 53 \end{pmatrix}$$

class vecteur;

class matrice

{ int M[3][3];

public: ~-

vecteur produit (const vecteur &);

~

{ ②

class vecteur

{ int M[3];

public: ~

friend vecteur matrice::produit (const vecteur &);

~

{ ②

Surcharge des opérateurs . S = A.gomme(B);

On souhaite écrire:

S = A + B ;

S = A.operator+(B) ;

Il suffit de remplacer dans le class tableau gomme
par operator +

class tableau

{ int *t, taille, nbelt;

public: 

tableau somme (const tableau &);



// ~~op~~ tableau operator+ (const tableau &);

4[9]



operator+

tableau tableau::somme (const tableau &v)

{ tableau res(taille);
for (int i=0; i<nbelt; i++)
{ res.t[i] = t[i] + v.t[i];

res.nbelt = nbelt;

return res;

}

Dans main :

S = A + B;

↑

$\Leftrightarrow S = B + A;$

~~opérateur~~

~~opérateur~~

~~opérateur~~

A, B.

A, B

A, B

x